

AppSec Core v1: A Formalized Normalization Ontology for Application Security

Pedro Farinha Independent Researcher pedro.farinha@shiftleft.pt

Supported by Shiftleft — Secure Software Engineering, lda.

Abstract

This paper publishes AppSec Core v1 as a formalized bounded ontology artefact for application security. *Bounded* here means: an extraction of the engineering substance of heterogeneous AppSec and regulatory sources (the development-and-maintenance practices a software engineer must satisfy on a pull request, sprint, or release at engagement scale), deliberately scoped to exclude governance substance (board reporting, vendor contractual risk management, regulatory examination response, workforce training programmes) which is routed to a complementary practitioner Manual surface. v1 is delivered as an OWL 2 DL ontology populated with 259 typed instances across the same ten domain slices introduced by AppSec Core v0 [1] (75 ControlObjectives, 69 Practices, 58 Mechanisms, 57 Artifacts), accompanied by a SHACL Core constraint set published as a complementary apparatus and validated under both the SHACL Core reference-implementation validator `pyshacl` and an in-house bounded-subset validator at parity for the apparatus's six ontology shapes. SHACL execution against the populated graph yields `sh:conforms = true` with zero `sh:Violation`. All artefacts are SHA-256-pinned at the cycle-close release tag chain.

The paper makes three contributions, each demonstrable by inspection of the published artefact and its validation outputs. (1) AppSec Core v1 as a formalized artefact — a citable, machine-validatable OWL 2 DL ontology and SHACL Core constraint set, replacing the YAML-only v0 surface. (2) Schema preservation under multi-source pressure — across the v0→v1 transition, the ten-slice partition, the four populated entity types, the EvidencePattern declarative class, and the cross-slice relation model are preserved; under expansion of the underlying source corpus from the five first-wave sources [2] to thirty-one sources at cycle close (a 6.2-fold source-count expansion absorbed by the design science cycle reported in [4]), the v0→v1 entity-count delta consists exclusively of additive instance population governed by an explicit protocol. (3) The AppSec Core Change Request (ACR) protocol with its explicit four-condition promotion threshold — the protocol is published with its threshold (multi-source convergence at ≥ 3 independent sources; multi-method convergence; practitioner-Manual content backing; slice fit) and is demonstrated through symmetric application across four worked decisions: ACR-001 *Secure Configuration Baseline Integrity* (promoted), ACR-002 *Security Requirements Lifecycle Management* (promoted), ACR-003 *Multi-anchor adjacency candidate batch* (not admitted), and ACR-004 *Output Rendering Safety / Context-Aware Encoding* (promoted). Three promotions and one non-admission decided under the same threshold provide initial evidence that the protocol produces falsifiable governance decisions rather than retrofitting outcomes; the discriminative power of the threshold against single-condition marginal failures is established by the threshold's specification, with empirically grounded near-miss cases registered as future work (§10.6).

The design science cycle that produced v1 from v0, the per-source corpus analysis, and the bidirectional validation of the practitioner Manual against v1 are out of scope and are reported in sibling papers [4, 5].

Keywords: ontology, application security, OWL 2, SHACL, normalization, governance protocol, AppSec

1. Introduction

1.1 Background and prior work

AppSec Core v0 was introduced in [1] as a normalized ontology for application security: ten domain slices with typed instances across four populated first-class entity types — ControlObjective, Practice, Mechanism, Artifact — plus an EvidencePattern declarative class indexed for verification semantics. v0 was derived bottom-up from a practitioner application-security Manual corpus and validated through canonical reduction of requirements drawn from a deliberately bounded set of five first-wave external sources: NIST Secure Software Development Framework (SSDF) [9], OWASP Application Security Verification Standard (ASVS) [10], Supply-chain Levels for Software Artifacts (SLSA) [11], CIS Critical Security Controls [12], and MITRE Common Attack Pattern Enumeration and Classification (CAPEC) [13]. The v0 paper [1] established the slice architecture, the four-type populated instance schema, and the per-slice contracts specifying scope, non-goals, and acceptance conditions; the companion compilation paper [2] applied v0 to a fifteen-chapter practitioner Manual case study under a coverage-preserving compilation method using a keyword-first normalization heuristic at cluster-level granularity.

Two limitations of v0 were declared in [1] and remained open:

1. Formalization. v0 was published as YAML-based typed instance schemas with per-slice contracts, sufficient for inspection by human readers and for mechanical processing under custom tooling, but not directly consumable by standard reasoners or by SHACL validators. The v0 paper noted OWL 2 / SHACL formalization as future work.
2. Source-set bounding. The five-source first wave was bounded by design — sufficient to demonstrate the ontology’s shape and the canonical-reduction property, but explicitly insufficient to claim that the slice partition would absorb a substantively wider corpus. Whether v0 would hold under expanded source pressure was an open empirical question.

This paper resolves both limitations. The ontology has been refined through a design science cycle (reported in [4]) under expansion of the underlying source corpus to thirty-one sources drawn from eleven organisational authorities and including AI-domain references, and the resulting AppSec Core v1 is published here as a formalized OWL 2 DL ontology with a SHACL Core constraint set. The cycle that produced v1 is not re-derived in this paper; v1 is presented as the cycle’s accepted output, with the cycle’s iteration trajectory, method refinement, per-source coverage analysis, and acceptance criterion reported separately in [4].

1.2 What this paper publishes

This paper publishes the v1 artefact and reports three contributions, each verifiable against the artefact and its validation outputs:

Contribution 1 — AppSec Core v1 as a formalized artefact. v1 is delivered as an OWL 2 DL ontology export (path under SHA-256 pinning at the cycle-close release tag) and a SHACL Core constraint set published as a complementary apparatus (§4). The ontology declares the same ten slices as v0 (§2), defines class hierarchies for the four populated entity types (ControlObjective, Practice, Mechanism, Artifact) plus the EvidencePattern declarative class with its controlled vocabularies (§3), and is populated with 259 typed instances. The SHACL apparatus comprises six schema-derived ontology shapes (regenerable from the canonical YAML schema by a published build script under SHA-256 pinning)

and five hand-maintained consumer-conformance shapes encoding model invariants. SHACL execution against the populated graph yields `sh:conforms = true` with zero `sh:Violation` under both the SHACL Core reference-implementation validator `pyshacl` (version 0.31.0 with `rdflib` 7.6.0) and an in-house bounded-subset validator at parity for the six ontology shapes (§4).

Contribution 2 — Schema preservation under multi-source pressure. The ten slices, the four populated entity types, the EvidencePattern declarative class, and the cross-slice relation model declared in v0 are preserved unchanged in v1 (§6). Under the corpus expansion absorbed by the cycle reported in [4] — from the five first-wave sources [2] to thirty-one sources at cycle close (a 6.2-fold source-count expansion; the cycle’s substrate at close records 3,861 source-items decomposed into 18,673 normalized claims) — the transition from v0 to v1 absorbed the expansion as additive instance population: per-entity counts grew (ControlObjective 70→75; Practice 63→69; Mechanism 48→58; Artifact 53→57; total typed instances 234 → 259, a +25 net delta), but no slice was added, removed, or repartitioned, no populated entity type was added, the EvidencePattern declarative class was retained without modification, and no cross-slice relation was added or modified. The v0→v1 entity-count delta decomposes by attribution to the cycle’s worked governance decisions (§6, §8, §9): thirteen additions through ACR-001’s promotion (+3 CO + 3 P + 3 M + 4 A), five through ACR-002’s promotion (+1 CO + 2 P + 2 M), three through ACR-004’s promotion (+1 CO + 1 P + 1 M), with four further within-slice refinement additions (Mechanism-level only) documented per-instance in the artefact’s release notes. Whether the schema continues to absorb additional expansion additively under future source pressure is an empirical hypothesis carried by the cycle’s reusable workflow [4]; this paper does not claim future-source bounding, only that the schema absorbed the cycle-close corpus as additive population.

The substantive engineering hypothesis the artefact embodies (forward-looking). C2’s evidence supports a stronger hypothesis the programme carries — that any new normative framework F (whether technical standard, regulatory regime, or industry guidance) decomposes into (a) claims mapping to existing v1 typed entities, normalising into actionable engineering substance, and (b) residual content routed to the Manual’s prose surface, without ontology schema disruption, modulo ACR-warranted additive amendments under the protocol of §7. This is not claimed as an established result of this paper. The framework-horizontal stress test — explicit decomposition against current and emerging regulatory frameworks (DORA, NIS2, CRA, GDPR, PCI-DSS, HIPAA, NIST SP 800-53, and analogous regimes) — is registered as future work for the programme; v1 provides the typed normalisation target the test will exercise, with [4]’s reusable workflow as the methodological discipline under which each absorption is governed.

Contribution 3 — The AppSec Core Change Request (ACR) protocol with its explicit four-condition promotion threshold, demonstrated by four symmetric worked decisions. The ACR protocol is the governance instrument by which candidate changes to the bounded ontology are admitted or refused. It is published in this paper because it governs the v0→v1 entity-count delta documented under C2 and because the four worked decisions are part of v1’s published content. The protocol’s threshold (§7) is stated in four conditions: (i) multi-source convergence — the candidate’s substance appears across at least three independent sources; (ii) multi-method convergence — the candidate is detected by independent normalization signals during the cycle’s re-application [4]; (iii) practitioner-Manual content backing — the candidate’s substance has a counterpart in the prescriptive practitioner Manual that complements the ontology; (iv) slice fit — the candidate fits within an existing slice’s contract or under a defensible additive amendment to that contract. Symmetric application across four cases — three promotions (ACR-001 *Secure Configuration Baseline Integrity* in §8.1; ACR-002 *Security Requirements Lifecycle Management* in §8.2; ACR-004 *Output Rendering Safety / Context-Aware Encoding* in §9.2) and

one non-admission (ACR-003 *Multi-anchor adjacency candidate batch* in §9.1) — establishes that the protocol’s threshold is testable rather than retrofitted: the same conditions produce non-promotion under measurement just as they produce promotion. The non-admission consumes its identifier as a discipline against governance-history compression.

These three are *factual* claims demonstrable by inspection of the published artefact, the published validation reports, and the four worked decisions. The paper does not absorb the design science cycle’s contribution as its own (the iteration trajectory, the method refinement from keyword-first to multistage, and the per-source coverage analysis are reported in [4]) and does not absorb the Manual-validation cycle’s contribution (the bidirectional validation of v1’s typed substrate against the practitioner Manual’s prose layer is reported in [5]).

1.3 The two surfaces of the programme architecture (context)

The artefact published here lives within the Security-by-Design Theory of Everything (SbD-ToE) programme architecture [3], which the reader needs to understand only at the level of background context for this paper’s claims. The SbD-ToE programme delivers application-security guidance through two complementary surfaces with asymmetric scope. The bounded ontology (this paper’s artefact) is a typed, validated, machine-readable subset of the application-security space, bounded by per-slice contracts and by the ACR governance protocol of §7, addressing the development-and-maintenance substance an engineer must satisfy at engagement scale (per pull request, per sprint, per release). The practitioner Manual is comprehensive: it carries both the engagement-view substance — a subset of which the ontology normalizes into typed entities, with the rest covered in prose throughout the Manual — and the organizational-view substance such as governance, training, vendor contractual security management, regulatory compliance regimes, and workforce programmes that lies outside the ontology’s bounds by design. The ontology is, in this sense, a normalized subset of part of what the Manual covers; the Manual is the comprehensive prescriptive corpus that complements the ontology in the programme architecture. This paper publishes the ontology surface; the bidirectional Manual-validation cycle is the subject of [5]. The downstream apparatus papers [6, 7, 8] consume both surfaces in combination as a security-requirements harness for code-generation consumers.

This paper does not need the reader to know more than the above about the Manual surface; the schema-preservation claim of C2 and the ACR protocol of C3 are stated at the ontology’s level. The third condition of the ACR threshold (practitioner-Manual content backing) references the Manual as a content-correctness checkpoint, not as a substantive contribution of this paper.

Engineering substance vs governance substance — the design-intentional split. The bounded ontology absorbs the engineering substance of normative AppSec sources (typically: secure development practices, technical safeguards, vulnerability management, incident detection, supply-chain integrity, configuration baseline integrity, identity and access controls, input/output validation, output encoding, threat modelling, security-event logging). The governance substance that frequently accompanies normative sources — board-level reporting obligations, vendor contractual risk management, regulatory examination response, workforce training programmes, organisation-level policy authorities, audit-firm interface — is routed by design to the Manual’s prose surface, not absorbed into the ontology’s typed entities. This split is intentional, not incidental: the ontology’s purpose is to provide a typed normalisation target for engineering substance an engineer (or LLM/MCP-grounded coding agent) can act upon at the application’s development scale; the governance substance complementing that engineering work has a different operational consumer (programme management, governance committees, organisational compliance functions) and lives in the Manual’s prose accordingly. The substantive claim of the pro-

gramme is therefore not “v1 substitutes reading the framework” — it is “v1 provides the normalisation target for the framework’s engineering substance, the Manual complements for the framework’s governance substance, and the ACR protocol governs where engineering substance pressures schema.”

Illustrative consumption shape (engagement scale). A representative consumption pattern: an engineer or coding agent receives a code change touching authentication middleware. Querying v1 with the change’s substantive surface returns a typed working set — for example, ControlObjectives in the Identity, Access & Session Trust slice (ACO–IAT–NNN family); their associated Practices (ACP–IAT–NNN); the supporting Mechanisms that operationalise those Practices (ACM–IAT–NNN); and the expected Artifacts (ACA–IAT–NNN) the change should produce or update for evidence. The engineer’s task collapses from “read the relevant chapters of every authority” to “demonstrate satisfaction of this typed working set against the diff under review”, with the Manual’s prose surface remaining available for the governance substance that does not normalise into typed entities. This consumption pattern is the *engagement-scale operationalisation* the ontology’s typing supports; the empirical evaluation of this consumption pattern under retrieval, language-model grounding, and grounded-versus-ungrounded compilation is reported in [6, 7, 8] respectively. This paper’s contribution is the publication of v1 as the typed reference layer; the operational evaluation lives downstream.

1.4 Scope

This paper publishes the v1 artefact and reports the three contributions above. The following are out of scope and are reported separately:

- *The design science cycle that produced v1 from v0* — its iteration trajectory, the method refinement (keyword-first → multistage normalization), the cross-source pressure analysis, the cycle’s *good-for-intended-fit* acceptance criterion and its evidence pillars, and the per-source-into-Reference mapping skin compatible with the NIST OLIR programme templates [32, 33] — is reported in [4].
- *Per-source corpus analysis* of the thirty-one sources from which v1’s instance population was drawn, including per-source convergence rates, per-source adjacency distributions, and override-mechanism logs — is reported in [4].
- *The bidirectional validation of v1’s typed substrate against the practitioner Manual’s prose layer* — including the Manual-coverage cycle, the edit-versus-rollback decisioning for each ACR addition, and the closure event that freezes corpus + Manual + ontology jointly for downstream consumption — is reported in [5].
- *Retrieval, large-language-model grounding, and experimental evaluation* against v1 — are addressed by the programme’s papers [6], [7], and [8].

The empirical case for v1 in this paper is restricted to what the artefact and its validation directly demonstrate: structural conformance (§4), schema preservation across the v0→v1 transition (§6), and the four-decision symmetric application of the ACR protocol (§7–§9). Claims of correctness for individual instance mappings, peer review of the per-source corpus, and stakeholder evaluation of v1’s downstream utility are open and registered as future work (§10).

Bounded engineering-ontology posture. AppSec Core v1 intentionally adopts a bounded engineering-ontology posture rather than attempting deep ontological realism, upper-ontology completeness, or richly inferential semantic-web research. The artefact’s purpose is to support operational normalization of heterogeneous AppSec sources at engineering scale (the harness consumer of [3, 6]), not to construct a foundational semantic theory of cybersecurity. In the terminology of Guarino [23], AppSec Core is an application ontology — domain- and task-specific, derivation-grounded rather than foundation-

grounded, with no commitment to upper-ontology categorial alignment (BFO, DOLCE, UFO). The OWL 2 DL ontology provides typed entities, an explicit cross-slice relation type system, controlled vocabularies, and the per-slice contracts that bound each slice’s scope and acceptance conditions; richer inferential commitments (deep subsumption hierarchies, role compositions beyond the declared properties, non-trivial logical entailment over the populated graph) are out of scope by design. The artefact is positioned as a bounded normalization ontology rather than as a candidate for semantic-web representational completeness.

Why an ontology rather than a typed knowledge graph. The choice of OWL 2 DL with SHACL apparatus rests on four substantive architectural requirements. First, the bounded ontology operates as a typed reference for heterogeneous-source normalization: each external source’s items are mapped against typed entities (ControlObjective / Practice / Mechanism / Artifact) under explicit per-slice contracts, and the OWL class hierarchy + property domain/range declarations make the typing computationally enforceable rather than convention-only. Second, the ACR governance protocol of §7 operates over declared structural commitments (slice partition, populated entity types, cross-slice relation type set, per-slice contracts); without OWL-level commitments, “schema preservation” would be a property of editorial discipline rather than of the artefact’s formal surface. Third, the SHACL apparatus of §4 enforces the cross-instance model invariants at a closed-world constraint layer complementing OWL’s open-world class semantics; this dual layering is operationally valuable for downstream tooling (validators, retrieval consumers) and is not satisfied by typed-KG conventions that lack a constraint-checking layer. Fourth, the bounded ontology is consumed by downstream MCP-grounded coding agents [8] and retrieval consumers [6, 7] requiring RDF/SPARQL-native serialisation with closed-world constraint enforcement at query time; this consumption profile is satisfied natively by OWL 2 DL + SHACL Core. Lighter-weight alternatives were considered — typed knowledge graphs (RDF + property-graph conventions without OWL semantics or SHACL constraint enforcement) and schema-only modelling layers (LinkML; JSON Schema bound to SHACL post-hoc) — but the four requirements above are not satisfied natively by these alternatives and would require non-trivial tooling supplementation; OWL 2 DL + SHACL Core was therefore considered the most appropriate form for the artefact’s normalization purpose, governance commitments, constraint-layer enforcement, and downstream consumption profile.

Positioning of v1. This paper does not claim that AppSec Core v1 is a complete or semantically final ontology of application security; it publishes a bounded, formally constrained, governance-controlled normalization ontology whose v0→v1 evolution absorbed the cycle-close corpus without schema restructuring.

1.5 Related ontology work and methodology positioning

AppSec Core v1 occupies a complementary layer to existing cybersecurity ontologies. The Unified Cybersecurity Ontology (UCO) [24] and the STUCCO ontology [25] integrate threat-intelligence schemas (CVE, CWE, CAPEC, STIX, and analogous structured-threat surfaces) for situational-awareness, threat-correlation, and intelligence-fusion use cases; recent cybersecurity knowledge-graph work continues to extend that surface [26]. AppSec Core targets a different operational consumer: the engineering-practice substance of heterogeneous AppSec and regulatory sources, normalized for development-and-maintenance compliance reuse at engagement scale (§1.3). The two ontology families address complementary layers of the cybersecurity-knowledge stack — threat intelligence (UCO/STUCCO/CKG) and engineering-practice normalization (AppSec Core) — and can be combined in downstream tooling without semantic conflict.

The cycle that produced v1 [4] instantiates an iterative ontology engineering workflow conceptually

aligned with the NeOn methodology [27] (scenario-based development with formal change control) and the LOT methodology [28] (industrial-oriented engineering with explicit governance instruments), with the ACR governance protocol of §7 providing the specific change-control instrument adapted to bounded normalization ontologies. The programme’s design science discipline follows the canonical DSR framework of Hevner et al. [20] and Wieringa [21] under the methodological reporting standards of Ralph et al. [22], with Peffers et al. [29] providing the operationalisation pattern most often cited in information-systems DSR.

1.6 Document structure

The rest of the paper is organised as follows. §2 describes the v1 slice and instance structure. §3 specifies the OWL 2 formalization. §4 specifies the SHACL apparatus and reports the validation outcome. §5 documents v1 as the accepted output of the cycle reported in [4]. §6 reports the schema-preservation evidence across the v0→v1 transition (Contribution 2). §7 specifies the ACR protocol and its four-condition promotion threshold (Contribution 3). §8 reports the worked decisions on candidates carried forward from the v0 baseline (ACR-001 and ACR-002, both promoted). §9 reports the worked decisions on candidates surfaced by the cycle’s re-application of the refined method (ACR-003, not admitted; ACR-004, promoted). §10 states limitations. §11 specifies the reproducibility surface. §12 lists references.

2. AppSec Core v1: structure and instance population

2.1 The ten-slice partition

AppSec Core v1 preserves the ten-slice partition introduced in v0 [1, §3]. v0’s partition was derived bottom-up from the practitioner Manual corpus through expert qualitative reading [1, §3.2], with each slice opened serially under an explicit per-slice contract specifying its scope, its non-goals, and its boundary conditions [1, §3.5]. Each slice has a declared primary control surface — the operational governance locus at which the slice’s controls converge in the delivered software lifecycle — captured by a categorical descriptor (`single_broad_control_surface`, `dual_control_surface`, `multi_control_convergent_surface`, `split_control_surface`).

The slice partition is presented in Table 1 with the v1 instance population per slice. Slice scope summaries are abridged from the per-slice contracts pinned at the cycle-close release tag; the contracts are the canonical reference for the full scope, non-goals, and boundary conditions of each slice.

Table 1. AppSec Core v1 slice registry, with scope summary and v1 typed-instance population. Slice scope summaries are abridged from each slice’s per-slice contract at the cycle-close release tag (`ontology-v1-final`); the contracts are the canonical reference for the full scope, non-goals, and boundary conditions of each slice. Slice abbreviation is the entity-prefix convention used throughout the ontology’s identifier scheme (e.g., `AC0-RPR-NNN` = `ControlObjective` in the RPR slice). Population per slice is the count of `ControlObjectives` + `Practices` + `Mechanisms` + `Artifacts` populated under the slice’s per-slice contract at the cycle-close release tag, read directly from `ontology-v1-final`’s instance index. Slice IDs follow the v0-published convention [1].

Slice ID	Slice abbreviation	Scope (abridged)	v1 population
ASC-01	SCBI	Supply Chain & Build Integrity (build pipeline integrity, source repository hygiene, dependency hygiene, container/image supply-chain controls)	30 (7 CO + 7 P + 6 M + 10 A)
ASC-02	IAT	Identity, Access & Session Trust (authentication, authorisation, session and identity boundary controls)	25 (7 + 6 + 6 + 6)
ASC-03	ATB	Architecture & Trust Boundaries (architectural trust boundary identification, sandboxing, isolation)	23 (7 + 7 + 5 + 4)
ASC-04	TSV	Testing, Security Validation & Empirical Assurance (SAST, DAST, penetration testing, security-relevant unit testing)	25 (7 + 7 + 5 + 6)
ASC-05	TMR	Threat Modeling, Risk Disposition & Mitigation Traceability (threat identification, risk disposition, mitigation traceability; ACR-002 additive amendment adds requirements-as-living-artefacts dimension per §8.2)	29 (8 + 9 + 8 + 4; includes ACR-002 additions)
ASC-06	SPC	Secret Handling, Protected Configuration & Operational Identities (secret material handling, protected configuration values, operational-identity discipline)	19 (7 + 6 + 4 + 2)

Slice ID	Slice abbreviation	Scope (abridged)	v1 population
ASC-07	IVF	Input Validation, Safe Parsing & Controlled Failure (input boundary discipline, safe parsing, controlled failure under hostile or malformed input; ACR-004 additive amendment expands scope to input/output validation and controlled failure per §9.2)	26 (8 + 7 + 5 + 6; includes ACR-004 additions)
ASC-08	ITS	Integration Trust & Service-to-Service Security (service-to-service authentication, integration boundary controls, trust delegation)	23 (7 + 6 + 5 + 5)
ASC-09	RPR	Release Promotion, Controlled Rollout & Rollback Readiness (release-promotion control, secure configuration baseline integrity, controlled rollout and rollback discipline; ACR-001 additive amendment adds baseline-integrity ControlObjectives per §8.1)	39 (10 + 9 + 8 + 12; includes ACR-001 additions)
ASC-10	SLG	Security Event Logging, Audit Trail & Centralized Logging (audit-log integrity, security-event recording, log retention and review)	20 (7 + 5 + 6 + 2)

Per-slice population counts are reported in the cycle-close release tag's slice registry artefact. The populated graph at v1 contains 259 typed instances total: 75 ControlObjectives + 69 Practices + 58 Mechanisms + 57 Artifacts.

2.2 The four populated entity types and the declarative EvidencePattern class

v1 preserves the four populated entity types declared in v0:

- ControlObjective: a reusable domain constraint or assurance goal, formulated in the verification voice (“the system shall...”);
- Practice: a reusable human or process discipline through which control objectives are realized;
- Mechanism: a concrete, enforceable means by which a practice is implemented (a tool, a config-

uration, a runtime control);

- Artifact: a control-relevant or evidence-bearing output (a document, a configuration record, a verifiable signed artefact).

Identifier convention. Every populated entity carries an identifier of the form `AC<T>--<SLICE>--<NNN>`, where:

- AC is the AppSec Core programme prefix;
- <T> is the entity-type initial: 0 for ControlObjective, P for Practice, M for Mechanism, A for Artifact;
- <SLICE> is the slice abbreviation per Table 1 (SCBI, IAT, ATB, TSV, TMR, SPC, IVF, ITS, RPR, SLG);
- <NNN> is a sequential three-digit per-type-per-slice index.

Examples: `AC0-IAT-001` is the first ControlObjective in the Identity, Access & Session Trust slice (ASC-02); `ACP-RPR-008` is the eighth Practice in the Release Promotion, Controlled Rollout & Rollback Readiness slice (ASC-09); `ACM-IVF-005` is the fifth Mechanism in the Input Validation, Safe Parsing & Controlled Failure slice (ASC-07); `ACA-SCBI-001` is the first Artifact in the Supply Chain & Build Integrity slice (ASC-01). Slice instances themselves use identifier `SLice<ASC-NN>` aligned with the slice IDs of Table 1.

Canonical source. The identifier scheme is declared canonically in the YAML schema source at the cycle-close release tag (`appsec-core-entity-schema-v0-draft.yaml` within the bundle's `ontology-v1-final` artefact, §11.1); the OWL TTL export (`appsec-core-v0-bounded-v1.ttl`, also within `ontology-v1-final`) instantiates the scheme as full IRIs under the `ac:` namespace, using the underscore form `AC<T>_<SLICE>_<NNN>` per TTL serialisation conventions (e.g., `ac:AC0_RPR_008` a `ac:ControlObjective`). The body prose of this paper uses the hyphenated form `AC<T>--<SLICE>--<NNN>` for readability; the two are semantically equivalent and identifier-stable across the bundle's serialisation formats.

A fifth class, `EvidencePattern`, was introduced in v0 as a supporting indexed class expressing the expected evidence shape for deterministic review of artefact compliance. v0 published a supporting index of 213 evidence-pattern entries [1, §6]. v1's populated graph contains zero `EvidencePattern` instances. What v1 preserves is the *EvidencePattern class declaration* — its formal class hierarchy, OWL property declarations, and SHACL `EvidencePatternShape` (with `sh:targetClass ac:EvidencePattern` plus optional property shapes referencing it from `ControlObjective` and `Artifact`) — at the apparatus level; the v0 supporting index of 213 patterns is preserved as a *separate artefact* within the cycle's deposited bundle (`appsec-core-evidence-pattern-index-v0.yaml`, see §11) and is consumable by downstream tooling that requires evidence-pattern semantics, but it is not migrated into v1's populated graph as first-class typed instances. The class is intentionally unpopulated at v1: the SHACL conformance report records `target_node_count = 0` for `EvidencePatternShape`. Population of the class as first-class typed instances at the populated graph is registered as future programme work (§10.4). The 234 / 259 figures reported throughout this paper for the v0 / v1 typed-instance counts therefore refer to the four populated entity types only and exclude the `EvidencePattern` declarative class.

2.3 The cross-slice relation model

v1 preserves the cross-slice relation model of v0 [1, §3.6]. The OWL ontology declares the relations under the canonical names exported by the build pipeline at `ontology-v1-final`: every populated `ControlObjective` is linked to at least one `Practice` (`objective_realized_by_practice`) and at least one `Mechanism` (`objective_implemented_by_mechanism`); `ControlObjectives` expect supporting `Artifacts` (`objective_expects_artifact`); slice membership is captured by `belongsToSlice`. The relation TYPES (`objective?practice`, `objective?mechanism`, `objective?artifact`, slice membership) and con-

nections between entity types are preserved unchanged from v0; the relation NAMES exported by the v1 OWL build pipeline use mixed convention (`belongsToSlice` camelCase short-form for slice membership; `objective_*_by_*` and `objective_expects_*` snake_case long-form for the typed inter-entity relations) consistent with the v1 OWL ontology release. No new cross-slice relation type has been added at v1.

Slice-membership functionality and cross-slice substantive coupling. `belongsToSlice` is functional: each populated entity (`ControlObjective` / `Practice` / `Mechanism` / `Artifact`) belongs to exactly one slice, expressed at the SHACL apparatus level as `sh:maxCount 1` on the property's shape. Substantive content that interacts across slices — for example, output rendering safety (ACR-004's substance) plausibly touches the IVF slice (input/output validation), the SCBI slice (build-time encoding configuration), and the ITS slice (service-to-service trust at the integration boundary) — is modelled via cross-slice relations (`objective↗practice`, `objective↗mechanism`, `objective↗artifact` triples that may cross slice boundaries), not via multi-slice membership. The canonical placement discipline is: each entity is placed in the slice of its primary control surface (the operational governance locus at which the entity converges in the delivered software lifecycle), with cross-slice substantive coupling expressed through the typed relations that explicitly connect entities across slice boundaries. This discipline avoids role conflation (the same entity playing structurally different roles across slices) by reserving the primary placement for the slice where the entity's primary control surface lives, and using the cross-slice relations as the operational language for inter-slice coupling. ACR-004's IVF placement instantiates this discipline: the substance's primary control surface is the input/output validation boundary at runtime; coupling to SCBI (build-time encoding configuration) and to ITS (service-to-service rendering boundaries) is expressed through `objective_implemented_by_mechanism` triples whose `Mechanism` instances may carry their own primary placement in those adjacent slices.

3. OWL 2 formalization

3.1 Class hierarchy

The OWL 2 DL ontology declares classes for the four populated entity types (`ControlObjective`, `Practice`, `Mechanism`, `Artifact`) and for the declarative `EvidencePattern` class, plus a `Slice` class whose individuals are the ten populated slices. The class hierarchy is shallow: each typed entity is a direct subclass of the top-level type, with no subtyping by slice (slice membership is captured by an object property, not by class hierarchy). The `Slice` class carries a `current_control_mode` data property whose values are restricted to the four controlled-vocabulary terms (`single_broad_control_surface`, `dual_control_surface`, `multi_control_convergent_surface`, `split_control_surface`). The ontology lies within the OWL 2 DL profile; classification and consistency checking are decidable in NEXPTIME under standard reasoner implementations (HermiT, Pellet, OpenLLET).

OWL 2 profile choice. The DL profile was selected over EL, QL, and RL on three grounds (note: all three OWL 2 sub-profiles support `rdfs:domain` and `rdfs:range` axioms; the differences across profiles concern axiom families that combine class and property expressivity in richer ways). (a) The artefact uses class disjointness and complement-style axioms for the populated entity-type partition (`ControlObjective` / `Practice` / `Mechanism` / `Artifact` / `EvidencePattern` as mutually disjoint classes) — EL excludes negation and disjointness; QL restricts class axiom forms in ways that would force lossy translation of the disjointness structure. (b) The artefact's reasoning use case is closed-world structural conformance checking at the SHACL apparatus layer (§4) rather than open-world query answering against very large ABoxes (which would favour QL); the OWL-level reasoning uses are restricted to class-hierarchy classi-

fication and consistency checking, well within DL expressivity. (c) The artefact is consumed by downstream tooling that combines OWL-level class-axiom reasoning with SHACL-level instance-graph constraint checking; DL provides the minimum expressivity that supports the disjointness axioms above without forcing translation to a profile whose restrictions are not motivated by the artefact's reasoning use cases.

Class-level disjointness at the vocabulary-value / entity boundary. Beyond the entity-type partition disjointness that motivates the DL profile choice above, v1 declares a class-level `owl:disjointWith` axiom between `ac:ControlledVocabularyValue` (the abstract grouping superclass for the six controlled-vocabulary classes — `ac:ObjectiveKind`, `ac:ObjectiveType`, `ac:ArtifactRole`, `ac:PracticeFamily`, `ac:MechanismFamily`, `ac:DetectableSurface`) and `ac:AppSecCoreEntity` (the abstract grouping superclass for the four populated entity types plus the `EvidencePattern` declarative class). The axiom formalises the categorical separation between vocabulary-value classes and AppSec Core entity classes at the OWL semantics level. It preserves OWL 2 DL decidability and is reachable to reasoners during class-hierarchy verification.

3.2 Object and data properties

The ontology declares object properties for the cross-slice relations (`belongsToSlice`, `objective_realized_by_practice`, `objective_implemented_by_mechanism`, `objective_expects_artifact`, plus the `EvidencePattern` indexing properties declared but not exercised at populated-graph level under v1). Data properties capture the per-instance descriptive content (item identifier, slice-scoped name, descriptive narrative, controlled-vocabulary tags). All properties are declared with explicit domain and range restrictions; cardinality constraints are expressed at the SHACL apparatus level (§4) rather than as OWL cardinality axioms, to keep the OWL ontology decidable under DL while expressing the populated-graph constraints in the SHACL layer where they are operationally consumed.

3.3 Controlled vocabularies

Controlled vocabularies cover (a) slice control modes (the four-term enumeration above), (b) per-instance landing-strength labels (`direct`, `strong`, `bounded`) used by downstream consumers to communicate the cycle's mapping confidence at instance level, and (c) cross-slice vocabulary terms (a curated set of approximately fifty domain-vocabulary terms covering the recurrent technical-anchor vocabulary across the AppSec slices). The controlled vocabularies are declared as OWL annotation properties with the term values pinned in the ontology; SHACL shapes (§4) enforce closed-world membership where required.

3.4 Instance population

At the populated graph for v1, the OWL ontology carries 259 typed instances across the four populated entity types as enumerated in §2.1. The bounded Turtle export (`ontology-v1-final`) serialises 1,970 data triples at the cycle-close release tag. Each instance has a unique IRI under the cycle's release tag namespace; relations between instances are explicit triples in the populated graph. Reasoner-decidable class hierarchy and property-domain/range checks pass under standard OWL 2 DL reasoners; the populated graph is consumable by reasoners that implement the OWL 2 DL profile.

4. SHACL formalization (the apparatus)

4.1 Apparatus composition

The SHACL apparatus accompanying v1 is composed of two complementary shape sets, an architectural pattern published as part of the cycle’s release [4]:

- Schema-derived ontology shapes (file: `appsec-core-v0-shapes.ttl` within the bundle’s apparatus artefact). Six `NodeShapes` covering the five typed entity classes (`Slice`, `ControlObjective`, `Practice`, `Mechanism`, `Artifact`) plus the `EvidencePattern` declarative class. These shapes are regenerable from the canonical YAML schema by a published build script (`build_shacl.py` at the same release tag); regenerating from the YAML under the documented build environment produces a deterministic SHACL output reproducible against the released shapes file (isomorphic up to blank node labeling under the RDFLib serialisation defaults of the documented environment).
- Consumer-conformance shapes (file: `consumer-conformance-shapes.ttl` within the bundle’s apparatus artefact). Five `ac:Claim`-typed shapes encoding model invariants that the populated graph must satisfy beyond the schema-derived shapes’ per-entity field-level constraints. These five shapes are hand-maintained from the model-invariant decisions recorded with the cycle [4] and capture: (M1’) slice-coherence invariants for each populated slice; (M3) consistency between `Practice` and `ControlObjective` populations within each slice; (M4) consistency between `Mechanism` and `ControlObjective` populations through the chain `Mechanism` → `Practice` → `ControlObjective`; (M4-card) claim well-formedness invariants; and (M4-card) claim target referential-integrity invariants. The hand-maintained shapes complement the schema-derived shapes at the populated-graph level: the schema-derived shapes verify that each instance carries its required fields under its type-level contract, and the hand-maintained shapes verify that the relations across instances satisfy the model invariants.

The apparatus composition is published under a single composed shapes graph at validation time; the two shape files compose without conflict.

4.2 Validation under two complementary validators

The populated graph is validated against the composed shapes graph under two complementary validators:

- The SHACL Core reference-implementation validator `pyshacl` at version 0.31.0 with `rdflib` 7.6.0. `pyshacl` implements the SHACL Core specification and is used here as the canonical validation oracle for the populated graph; the implementation is community-maintained against the W3C SHACL Recommendation rather than W3C-endorsed.
- An in-house bounded-subset validator at parity with `pyshacl` for the apparatus’s six ontology shapes. The bounded-subset validator was developed as a programme-internal tool predating the apparatus’s `pyshacl`-canonical release and has been retained as a parallel validation surface for tooling that consumes per-shape granular validation outputs (the bounded-subset validator emits per-shape conformance signals usable by downstream tooling without parsing the full SHACL conformance report). Parity with `pyshacl` is verified at apparatus release time across the six ontology shapes and is preserved within the bundle’s apparatus artefact.

The composed SHACL shapes graph carries 396 shape triples across 11 `NodeShapes` (the six schema-derived ontology shapes plus the five hand-maintained consumer-conformance shapes) at the cycle-close release tag. Validation against the populated graph at v1’s cycle-close release tag yields:

- `pyshacl 0.31.0 + rdflib 7.6.0: sh:conforms = true; zero sh:Violation` across the composed

shapes graph.

- Bounded-subset validator: `conforms = true`; zero violations across the six ontology shapes (parity scope).

The two validators agree at v1’s cycle-close. Where the two diverge in scope, `pyshacl` is the canonical oracle for the populated-graph claim; the bounded-subset validator’s per-shape granular outputs are auxiliary to consumer tooling.

4.3 What the validation evidence supports

The validation outcome supports the structural-conformance subset of Contribution 1: at the populated graph for v1, the composed SHACL apparatus reports conformance under both validators. This is a binary structural conformance claim. It does *not* claim that any individual mapping decision recorded against an instance is content-correct against the source authority that motivated the instance — content-correctness of individual mappings is the territory of subject-matter-expert review, registered as future work (§10). The SHACL conformance verdict supports that the populated graph satisfies the apparatus’s structural constraints; it does not certify the substantive correctness of the apparatus’s constraints themselves or of the instances’ substantive content.

Shape-set completeness disclosure. The apparatus’s evidentiary value depends on the completeness of the shape set against the invariants the artefact intends to enforce. The six schema-derived ontology shapes cover field-level structural constraints per entity type (required fields, controlled-vocabulary value ranges, identifier conventions, basic cardinality); the five hand-maintained consumer-conformance shapes encode the cross-instance model invariants identified during the cycle’s design [4]. Completeness of the shape set against *all* possible invariants of a well-formed AppSec Core ontology is not claimed: specifically, semantic-level invariants (e.g., whether a promoted `ControlObjective`’s verification voice is well-formed; whether a `Practice`’s description is implementable; whether a `Mechanism`’s enforcement is operationally testable) are not expressible in SHACL Core and are not captured by the apparatus. The conformance verdict therefore supports structural well-formedness against the explicit shape set as released, not semantic correctness or comprehensiveness of the invariant set itself; expansion of the invariant set to capture additional semantic constraints is registered as future work (§10).

5. The accepted output of the design science cycle

v1 is published here as the accepted output of the design science cycle reported in [4]. The cycle reported in [4] iterates the v0 ontology and a v0-era keyword-first compilation method [2] across three iterations under the pressure of expansion to thirty-one sources, refining method (Iteration 1), exercising the ACR protocol of §7 (Iteration 2), and pressure-testing the bounded thesis under an extension to the AI/ML domain (Iteration 3). The cycle’s acceptance criterion — *good-for-intended-fit*, operationalised through four evidence pillars covering structural conformance under SHACL Core (Pillar 1), per-source coverage acceptable under independent third-party cross-references (Pillar 2 — oracle agreement), multi-mode evaluation returning no failure-class signal (Pillar 3), and rejection of random source-claim assignment with semantic-specialisation direction under permutation-test null model (Pillar 4 — k-way structural concentration) — is reported in [4]; the criterion’s first pillar is satisfied by §4’s SHACL conformance verdict.

The artefact reported in this paper is the cycle’s ontology-side output at cycle close: the OWL 2 DL ontology of §3, the SHACL apparatus of §4, and the populated graph at v1’s cycle-close release. The

artefact is SHA-256-pinned within the cycle’s deposited bundle (§11.0) under the stable artefact identifiers `ontology-v1-final` and `apparatus-shacl-pyshacl-v3`. The cycle’s narrative is not re-derived in this paper; the artefact is published as the cycle’s accepted output, with citation to [4] for the cycle that produced it.

6. Schema preservation under multi-source pressure

6.1 What is preserved across $v0 \rightarrow v1$

The following elements of the $v0$ schema are preserved unchanged in $v1$:

- The ten-slice partition declared in $v0$ [1, §3]. No slice was added, removed, or repartitioned at $v1$; the `current_control_mode` of each slice is preserved.
- The four populated entity types (ControlObjective, Practice, Mechanism, Artifact). No populated entity type was added.
- The EvidencePattern declarative class. The class declaration is preserved at the OWL level; $v1$ does not populate additional EvidencePattern instances at this release.
- The cross-slice relation model: `belongsToSlice`, `objective_realized_by_practice`, `objective_implemented_by_mechanism`, `objective_expects_artifact` (canonical names exported by the $v1$ OWL build pipeline at `ontology-v1-final`; relation TYPES and connections between entity types are preserved unchanged from $v0$, see §2.3). No new cross-slice relation type was added or modified.
- The per-slice contracts specifying scope, non-goals, and acceptance conditions. $v1$ preserves all $v0$ contracts unchanged, with one exception: the IVF slice’s contract is additively amended through ACR-004’s promotion (§9.2), expanding scope from “input validation and controlled failure” to “input/output validation and controlled failure” without modifying or removing prior scope, non-goals, or acceptance conditions.

Additive OWL-level structural rigour at $v1$ cycle-close. Beyond preservation of the schema elements above, $v1$ ’s cycle-close state includes OWL-level structural axioms and metadata not expressed in $v0$ ’s YAML-only serialisation: the class-level `owl:disjointWith` axiom between `ac:ControlledVocabularyValue` and `ac:AppSecCoreEntity` (§3.1), ontology-header `dcterms` metadata declared at the ontology IRI (publisher, source, issued date, status, depiction), and per-instance `dcterms` metadata declared at the populated-instance IRIs. These refinements are structurally additive at the OWL level and do not alter any preservation claim above: typed-instance counts are preserved (Table 2), the slice partition is preserved, cross-slice relation types are preserved, and per-slice contracts are preserved. The refinements have zero effect on the apparatus’s SHACL conformance verdicts under both validators (§4.2) and zero effect on the cycle’s substrate-grounding and embedding-similarity tooling [4].

6.2 What is added across $v0 \rightarrow v1$: additive instance population governed by the ACR protocol

The corpus expansion absorbed by the cycle reported in [4] is reflected in $v1$ entirely as additive instance population under the typed entity schema. The $v0 \rightarrow v1$ entity counts below are sealed by per-identifier audit between the $v0$ frozen state (released as the `ontology-v0-frozen` artefact in the cycle’s bundle, §11) and the $v1$ final state (released as `ontology-v1-final`); the per-entity verification trail is preserved in the bundle’s audit-trail surface.

Table 2. AppSec Core $v0 \rightarrow v1$ typed-instance counts. $v0$ counts are read directly from the `ontology-`

v0-frozen artefact’s instance index (appsec-core-v0-instance-index.yaml); v1 counts are read directly from the same file at the ontology-v1-final artefact. The v0 published surface in [1] enumerates the same 234 typed instances; v0 cycle-entry equals v0 published (no within-slice refinement pre-iteration).

Entity type	v0 [1]	v1 cycle-close	Net delta
ControlObjective	70	75	+5
Practice	63	69	+6
Mechanism	48	58	+10
Artifact	53	57	+4
Total typed instances	234	259	+25

Slice count is preserved at 10 across v0 and v1.

6.3 Attribution of the delta to ACR-protocol decisions and within-slice refinement

The +25 delta decomposes by attribution to the cycle’s worked decisions under the ACR protocol of §7 and to a single within-slice 4-Mechanism refinement patch. The per-source attribution below is sealed by per-identifier audit between the ontology-v0-frozen and ontology-v1-final bundle artefacts; the per-identifier introduction record is preserved in the bundle’s audit-trail surface (§11).

The per-identifier introduction record (which cycle artefact-state introduced each new entity identifier and its associated relation declarations) is preserved in the bundle’s audit-trail surface; this section enumerates the identifiers per source and Table 3 summarises the per-source attribution net counts.

The identifiers introduced under each attribution source are:

- ACR-001 (RPR slice): ACO-RPR-008, ACO-RPR-009, ACO-RPR-010; ACP-RPR-008, ACP-RPR-009, ACP-RPR-010; ACM-RPR-008, ACM-RPR-009, ACM-RPR-010; plus four supporting RPR Artifacts.
- ACR-002 (TMR slice): ACO-TMR-008; ACP-TMR-008, ACP-TMR-009; ACM-TMR-007, ACM-TMR-008.
- ACR-004 (IVF slice): ACO-IVF-008; ACP-IVF-007; ACM-IVF-005.
- Within-slice refinements (4-Mechanism patch): ACM-RPR-005 (release-promotion); ACM-SCBI-006 (software-composition-and-build-integrity); ACM-SLG-005, ACM-SLG-006 (secure-logging).

Table 3. v0 → v1 entity-count delta — per-source attribution.

Source of addition	+CO	+P	+M	+A	Total
ACR-001 <i>Secure Configuration</i> <i>Baseline Integrity</i> (RPR slice ASC-09; §8.1)	+3	+3	+3	+4	13

Source of addition	+CO	+P	+M	+A	Total
ACR-002 <i>Security Re-requirements Lifecycle Management</i> (TMR slice ASC-05; §8.2)	+1	+2	+2	0	5
ACR-004 <i>Output Rendering Safety / Context-Aware Encoding</i> (IVF slice ASC-07; §9.2)	+1	+1	+1	0	3
Within-slice refinements (4-Mechanism patch)	0	0	+4	0	4
Total v0 → v1 net	+5	+6	+10	+4	+25

The decomposition reconciles bit-identically against the totals in Table 2 (+5 CO + 6 P + 10 M + 4 A = +25; v0 234 → v1 259). No entity removals occurred between v0 and v1; identifier stability is preserved (every entity identifier in v0's instance index is preserved in v1). The within-slice 4-Mechanism patch is a single coherent change to existing slice contracts: ACM-RPR-005 refines an existing ControlObjective in the release-promotion slice without expanding scope; ACM-SCBI-006 refines existing scope in supply-chain-and-build-integrity; ACM-SLG-005 and ACM-SLG-006 refine the security-event-logging slice. None of the four required ACR review under §7's threshold because none was cross-source-convergent at the protocol's bar (≥ 3 independent sources) and none was scope-amending at the slice level — each refines within existing contract scope and population. Their authority for inclusion is the slice's existing per-slice contract; the cycle's per-source curation [4] surfaced them as instance-level refinements consistent with those contracts.

Within-slice refinement governance discipline. Within-slice refinements that do not meet the ACR threshold (§7.2) are admitted under a lighter governance discipline than ACR-level changes, with three constraints: (a) each refinement is recorded per identifier in the artefact's release notes and is verifiable against the bundle's audit-trail surface (§11); (b) each refinement must remain consistent with the slice's existing per-slice contract — within-slice refinements cannot amend slice scope, non-goals, or acceptance conditions (any candidate that requires such amendment is escalated to ACR review under §7); and (c) cumulative within-slice refinements that exceed a defined volume threshold between major releases are escalated to ACR-level review to guard against silent slice-scope drift. The volume threshold (registered as future programme work: provisionally ≥ 5 within-slice refinements per slice or ≥ 20

corpus-wide between major releases) operationalises the guard against backdoor scope amendment via aggregation. The four within-slice Mechanism additions documented above remain well below this provisional threshold and are recorded for per-identifier audit at the cycle-close release.

6.4 Verification trail

The per-source attribution in Table 3 is verifiable per identifier against the bundle’s audit-trail surface (§11), which records — for each of the 25 net additions — the cycle artefact-state at which the identifier was introduced and the corresponding relation declarations introduced alongside it. A reader auditing the $v0 \rightarrow v1$ transition against the published surface of [1] can reconstruct Table 3’s per-source attribution from the audit-trail surface alone; the audit trail is reproducible against the bundle’s ontology- $v0$ -frozen and ontology- $v1$ -final artefact states (§11).

6.5 The empirical claim

The empirical claim of Contribution 2 is that, at the cycle-close corpus of thirty-one sources (a 6.2-fold source-count expansion from the five-source first wave [2]), the $v0$ schema (slice partition + populated entity types + EvidencePattern declarative class + cross-slice relation model + per-slice contracts) admitted the corpus pressure as additive instance population, with the entity-count delta governed by the ACR protocol’s worked decisions and a documented set of within-slice refinements. The claim is bounded to the cycle-close corpus; whether the schema continues to absorb additional expansion additively under future source pressure is an empirical hypothesis carried by the cycle’s reusable workflow [4], not a property guaranteed by the artefact published here.

Source-independence caveat. The cycle-close corpus should not be interpreted as thirty-one fully orthogonal semantic systems; several sources belong to partially overlapping governance ecosystems. The OWASP family — ASVS, SAMM, DSOMM, Proactive Controls, and the OWASP Top-10 specialisations — shares editorial framing and convergent vocabulary even where individual documents target distinct surfaces. The NIST family (SSDF, SP 800-53, AI 100-2, AI RMF) is similarly co-authored within a single organisational authority. The ACR protocol imposes no organisational-authority gate — authority spread is a descriptive property of the corpus, not a promotion condition (§7.2 Condition 1) — so the threshold does not mitigate this dependency: partial vocabulary co-evolution within and across authorities remains a property of the corpus that multi-source convergence does not eliminate. The substantive claim of C2 is therefore that the $v0$ schema absorbed the corpus’s *partially-overlapping* multi-source pressure as additive instance population, not that it absorbed thirty-one independent semantic systems.

Regulatory horizontal sources in the cycle-close corpus. The corpus includes horizontal regulatory frameworks alongside the AppSec-canonical and AI/ML extensions: PCI DSS v4.0.1 and PCI SSLC v1.1 (PCI SSC); EU CRA, EU NIS2, EU GDPR, EU DORA (EU regulatory wave); HIPAA Security Rule (US HHS). Per-source claim density to $v1$ typed entities varies substantially across these sources — PCI DSS contributes high-density mappings (e.g., 104 claims to the RPR slice’s ACR-001-promoted ControlObjective set alone; substantial mappings across IAT, IVF, SCBI), consistent with PCI DSS’s heavy engineering-substance loading. EU horizontal regulations (CRA, NIS2, GDPR, DORA) and HIPAA contribute lower-density mappings, consistent with their governance-substance loading: bulk of these regulations’ content is governance substance (board-level reporting, vendor risk management, regulatory examination response, organisation-level policy authority, workforce training programmes) routed by design to the Manual’s prose surface (§1.3); the engineering substance fraction that *does* normalise to $v1$ typed entities (technical safeguards, vulnerability management dispositions, incident detection mechanisms, supply-chain integrity controls) maps coherently into the existing slice partition without requiring schema disruption. This empirical pattern across the regulatory subset is consistent

with the engineering-substance / governance-substance design split declared in §1.3. The framework-horizontal stress test — explicit decomposition against each regulatory framework with per-framework engineering-substance / governance-substance fractioning — is registered as future programme work; the present paper reports the cycle-close corpus’s absorption pattern at the aggregate level.

A potential failure mode of the schema-preservation claim — what would have counted as failure under the cycle’s measurement — is enumerated explicitly: a slice added or removed; a populated entity type added; a cross-slice relation added or modified; a per-slice contract restructured (rather than additively amended); an ACR candidate that the protocol’s threshold could not bound under any of (i) within-existing-slice-amendment, (ii) within-existing-population-refinement, or (iii) explicit non-admission with identifier consumption. None of these failure modes occurred at the cycle-close corpus. The schema-preservation claim is anchored to the absence of these failure modes, not to a generic claim of stability.

Anti-tautological defense. The ACR protocol’s Condition 4 (§7.2) excludes partition-level restructuring — a sibling-slice repartition or addition of a new populated entity type — from the protocol’s scope; such a candidate would require a separate partition-level governance exercise. This design choice makes “no slice repartition” a structural property of the protocol rather than an empirical observation: the protocol cannot, by construction, *produce* partition-level disruption from within. The schema-preservation evidence of Contribution 2 is therefore: *across the cycle-close thirty-one-source corpus and the four worked ACR decisions, no candidate’s substance required partition-level restructuring to be normalized*. If a candidate had emerged whose substance could not be admitted under any of the in-scope ACR dispositions (within-existing-slice-amendment, within-existing-population-refinement, or explicit non-admission with identifier consumption), the protocol would have flagged it as out-of-scope, requiring a separate partition-level governance exercise — a path the cycle did not need to invoke. The substantive claim is therefore the *absence of partition-level pressure* under the cycle-close corpus, not the protocol’s inability to handle such pressure if it arose. Whether future expansion produces a candidate that requires partition-level governance is an empirical question carried by the cycle’s reusable workflow, not foreclosed by the protocol’s design.

Out-of-scope candidate register (the audit-trail counterpart). To make “no partition-level pressure emerged” verifiable rather than a structural absence, the protocol’s audit-trail surface (§11) records — for every candidate the cycle considered — its disposition under §7.2 Condition 4: in-scope (admitted to one of the three in-scope dispositions: promotion, within-population refinement, or non-admission with identifier consumption) or out-of-scope (classified as requiring partition-level governance, with the reason recorded). At cycle close, the out-of-scope register contains $N = 0$ entries: no candidate the cycle considered was classified as out-of-scope of the protocol’s Condition 4. This zero-entry register is the verifiable counterpart of the substantive claim — the absence is empirical, not constructive. The register is preserved in the bundle’s audit-trail surface alongside the four in-scope worked decisions of §§8–9.

Relation to conservative extension in the ontology-engineering literature. The schema-preservation claim of Contribution 2 is *weaker* than conservative extension *stricto sensu* [30, 31]: it does not claim preservation of entailments over the v0 vocabulary, but claims that (i) the v0 TBox (slice partition + populated entity type declarations + cross-slice relation type set) is preserved unchanged in v1; (ii) the cross-slice relation type system is unchanged; and (iii) the v1 ABox contains the v0 ABox under identifier stability (every v0 entity identifier is preserved in v1 with its full property set carried forward). The relation to module-theoretic ontology evolution (locality-based module extraction; formal conservative-extension verification under the Σ -conservative-extension definition) is registered as future programme

work; the claim reported here is the weaker structural-and-identifier preservation under additive ABox population.

7. The AppSec Core Change Request (ACR) protocol

7.1 What an ACR is

An AppSec Core Change Request (ACR) is a candidate change to the bounded ontology submitted under the protocol described in this section. Candidates may be:

- a candidate new ControlObjective, Practice, Mechanism, or Artifact;
- a candidate new cross-slice relation;
- a candidate scope amendment to a slice’s per-slice contract (additive amendment within bottom-up derivation discipline);
- a candidate non-admission decision recording that a candidate’s substance does not meet the protocol’s threshold and is held outside the ontology, with the reason recorded.

An ACR receives a unique identifier of the form ACR-NNN, assigned at submission time, and consumes that identifier permanently regardless of outcome (promotion, non-admission, or any future re-evaluation under a different threshold). The identifier-consumption discipline preserves a complete record of every governance decision the protocol has produced, against which future submissions can be audited.

7.2 The four-condition promotion threshold

The protocol’s threshold for promotion is stated as four conditions, all of which must be met for a candidate to be admitted to the ontology:

Condition 1 — Multi-source convergence. The candidate’s substance appears across at least three independent sources. The burden is asymmetric in the strength of the claim the change asserts: a promotion that introduces new Core entities (a ControlObjective and its realizing chain, or Practices and Mechanisms, optionally with an additive contract amendment) carries the burden of ≥ 3 independent sources; a change that would open a new engineering slice carries the extraordinary burden of ≥ 4 independent sources plus a demonstration of structural inadequacy in the existing partition — the partition-level case routed under Condition 4 to a separate governance exercise. The three-source bar is calibrated against the cycle’s first-wave bounding test [2]: at three independent sources, convergence on the same substance establishes that the candidate is not idiosyncratic to a single team’s interpretation. Convergence below three sources is a detection-level signal that may motivate further investigation but does not on its own meet the promotion bar. Organisational-authority spread is a descriptive property of the corpus, not a promotion condition: the threshold counts independent sources, and authority diversity is reported where it bears on interpretation but never gates admission. The threshold is governance-calibrated rather than mathematically optimal: it operationalises the trade-off between admitting clusters carried by single-pattern editorial framing (a lower threshold would do this) and excluding substantively important but distinctively-framed concepts (a higher threshold would do this), at a heuristic balance point anchored to the first-wave bounding test. Sensitivity analysis under alternative thresholds (a two-source minimum; a higher-source bar) is registered as future work (§10).

Condition 2 — Multi-method convergence. The candidate is detected by independent normalization signals during the cycle’s re-application of the refined method [4]: a domain-first signal reading the source’s own structural hierarchy to place the candidate at a coherent slice, a sentence-embedding-

similarity signal returning a coherent top-anchor cluster across the convergent sources, and an explicit override-mechanism record where the author’s curated decision concurs with or overrides the method-converged signal under audit-traceable discipline. All three signals are required; method-single-signal detection does not on its own meet Condition 2.

Condition 3 — Practitioner-Manual content backing. The candidate’s substance has a counterpart in the prescriptive practitioner Manual that complements the bounded ontology in the programme architecture [5]. Where the Manual does not yet cover the substance, the cycle’s gap-detection [4] prompts the Manual’s author to fill the missing content before the candidate proceeds; ontology promotion never precedes Manual-prose authorship of the substance. The Manual’s authoring is iteratively responsive to ACR pressure rather than antecedent to it: a candidate surfacing from source convergence may trigger gap-detection against the Manual, which the Manual’s authoring process then resolves; ontology promotion under §7 follows Manual-prose authorship, but the Manual itself evolves under cycle pressure (the bidirectional Manual-validation discipline is reported in [5]). The condition’s purpose is to keep ontology change tied to authored prescriptive content rather than to source-side convergence alone.

Condition 4 — Slice fit. The candidate fits within an existing slice’s per-slice contract (its scope, non-goals, acceptance conditions), or motivates a slice-contract amendment whose acceptance under the bottom-up derivation discipline can be defended without restructuring sibling slices. Restructuring a sibling slice or repartitioning the ten-slice partition is outside the protocol’s scope; such a candidate would be a partition-level decision requiring its own (separate) protocol exercise.

7.3 Symmetric application: promotion or non-admission

The protocol is symmetric: under the same four-condition threshold, a candidate may be promoted (admitted to the ontology, with the promoted entities entered at the populated graph) or not admitted (held outside the ontology, with the reason recorded against whichever condition failed and the identifier consumed). Symmetric application of the same conditions to both outcomes is the protocol’s mechanism for preventing retrofit: if non-promotion never occurred under the protocol, the threshold would be unfalsifiable. Section 9.1 reports the cycle’s first non-admission decision (ACR-003), which alongside the three promotions of §§8.1, 8.2, and 9.2 provides initial evidence that the threshold produces falsifiable governance decisions; near-miss non-admission cases that would test single-condition marginal failures are registered as future work (§10.6).

7.4 The protocol’s evidence trace

For each ACR (admitted or not), the protocol records: (a) the candidate’s substance and proposed entities or scope amendment; (b) the per-condition evidence (the convergent sources for Condition 1; the per-stage method signals for Condition 2; the Manual content reference for Condition 3; the slice contract reference and amendment proposal if any for Condition 4); (c) the disposition (promotion, non-admission); (d) where promoted, the assigned entity identifiers and the slice-contract delta if any; (e) where not admitted, the reason against the failed condition. The evidence trace is recorded against the artefact’s release notes at the cycle-close release tag, with each ACR’s per-condition evidence verifiable against the cycle’s published per-source records [4].

8. Worked decisions on candidates carried forward from the v0 baseline

8.1 ACR-001 — Secure Configuration Baseline Integrity (promoted)

Substance. The integrity of secure configuration baselines as a first-class engineering practice in the release-promotion-controlled-rollout-and-rollback-readiness slice (ASC-09 / RPR family): the existence of an authoritative secure-baseline configuration for security-relevant runtime parameters, the integrity of security-relevant configuration values against unauthorized override, and the discipline of baseline review with exception visibility and change discipline. Configuration baseline integrity governs what is deployed to runtime and is therefore canonically placed in the release-promotion slice, which carries the release-time controls that gate baseline integrity at promotion and rollback boundaries.

v0-era status. The substance was already present in the v0 Manual mapping [1, §10] as CFG-001→007 (“secure configuration baselines... documented in the corpus in 2023”) with anteriority noted relative to ASVS v5.0’s 2025 formalisation of `secure_configuration_baseline`. v0’s published surface reflected the substance under the Manual-mapping label without first-class ControlObjective status under an explicit governance protocol.

Per-condition evidence. - *Condition 1 (multi-source convergence at ≥ 3 independent sources)*. At promotion time the substance converges across 4 independent sources — NIST SSDF PW.9, OWASP ASVS V13, CIS Control 4, and MITRE CWE-1188/276/426–428 — meeting the protocol’s ≥ 3 -source bar for a new-Core-entity promotion. Configuration baseline integrity is a near-universal AppSec topic and the broader corpus carries it widely, but promotion rests on the four named convergent sources rather than on corpus-wide spread. The full per-source claim distribution is preserved in the substrate’s per-pilot configuration files at the cycle-close release tag (see [4 §15.2]). - *Condition 2 (multi-method convergence)*. The cycle’s re-application of the refined method [4 §4] returns convergent signals across the structural-hierarchy stage (the source-side ancestors place the substance within the RPR slice), the embedding-similarity stage (top-anchor cluster coheres across the convergent sources), and the override-mechanism record (no override required against the method-converged signal). - *Condition 3 (practitioner-Manual content backing)*. The substance is present in the practitioner Manual under the release-promotion and configuration-management chapters of the Manual’s prose layer; the Manual’s coverage of secure configuration baseline substance has been authored at the Manual-validation cycle’s documented state [5]. - *Condition 4 (slice fit)*. The candidate fits within the existing RPR slice’s per-slice contract; no slice-contract amendment is required.

Disposition. Promoted. The substance is admitted to the ontology as three new ControlObjectives entered at the populated graph for v1: ACO-RPR-008 (Secure Configuration Baseline Integrity), ACO-RPR-009 (Security-Relevant Configuration Integrity And Override Control), and ACO-RPR-010 (Baseline Review, Exception Visibility And Change Discipline), together with their associated Practice and Mechanism chains.

8.2 ACR-002 — Security Requirements Lifecycle Management (promoted)

Substance. The discipline of managing security requirements as living artefacts across the application’s development cycle: requirements creation, requirements review, requirements traceability across sprint boundaries, requirements deprecation, and requirements-versus-threat coherence under change. Distinct from the threat-side substance the TMR slice (ASC-05) already carries (threat identification, threat-and-risk traceability to mitigation, mitigation prioritisation), the candidate anchors the requirements-as-living-artefacts dimension of the TMR slice.

v0-era status. The substance was identified at v0 closure as a deferred candidate; v0’s published pro-

tol thresholds were not applied symmetrically across the v0-era candidate set [1], and ACR-002 sat at v0 closure in a “candidate, not yet decided under protocol” state. Security-requirements lifecycle management is so foundational to AppSec that its absence as a first-class ControlObjective set in v0 deserves explanation. The *substance* was already present in the practitioner Manual’s prose layer at v0 publication (Manual mapping label REQ-LCM-* per [1, §10]); what ACR-002 promotes to v1 is first-class typed-entity status under an explicit governance protocol, not the substance itself. The promotion is therefore an *artefact-formalization step* — moving substance from Manual prose into the bounded ontology under the ACR protocol’s four-condition discipline — rather than a discovery of new substance unaccounted for in the v0-era programme.

Per-condition evidence. - *Condition 1.* At promotion time the substance converges across 5 independent sources — NIST SSDF, OWASP SAMM (Security Requirements practice [16]), PCI SSLC, NIST SP 800-53, and SAFECode Fundamental Practices for Secure Software Development — meeting the protocol’s ≥ 3 -source bar for a new-Core-entity promotion. The broader corpus carries the substance more widely, but promotion rests on the five named convergent sources. The full per-source claim distribution is preserved in the substrate’s per-pilot configuration files at the cycle-close release tag (see [4 §15.2]). - *Condition 2.* Multi-stage method convergence: the structural-hierarchy stage places the candidate within the TMR slice; the embedding-similarity stage returns coherent top-anchor cluster across the convergent sources; the override-mechanism record concurs with the method signal. - *Condition 3.* Substance present in the practitioner Manual under the threat-modeling chapter of the Manual’s prose layer. - *Condition 4.* The candidate motivates an additive amendment to the TMR slice’s per-slice contract scope, declaring the requirements-as-living-artefacts dimension as within scope alongside the existing threat-side substance. The amendment is additive: prior scope, non-goals, and acceptance conditions are preserved.

Substantive question raised by the candidate. *Does ACR-002 duplicate scope already in the TMR slice?* The TMR slice at v0 closure carries the threat-side substance — threat identification, threat-and-risk traceability, mitigation prioritisation — with security requirements treated as the *output* of the threat-modeling activity rather than as a governance discipline in their own right. The candidate’s substance is the *requirements-as-lifecycle-artefact* dimension, distinct from the threat-side dimension v0 already captured. Three considerations support the additive amendment rather than collapse into the existing threat-side anchor: the convergent sources surface the candidate substantively as requirements lifecycle (a temporal-and-traceability dimension distinct from the threat-side dimension); v0’s slice-contract before the amendment did not declare a primary anchor for the requirements-as-living-artefacts dimension; and the cycle’s per-stage method signals support the additive amendment’s slice fit without restructuring the threat-side substance. The promoted ControlObjective therefore extends the slice along an orthogonal dimension to the existing threat-side substance, rather than duplicating it.

Disposition. Promoted. The substance is admitted to the ontology as one new ControlObjective + two new Practices + two new Mechanisms in the TMR slice, together with the additive amendment to the slice’s per-slice contract scope.

9. Worked decisions on candidates surfaced by cycle re-application

9.1 ACR-003 — Multi-anchor adjacency candidate batch (not admitted)

Substance. A batch of approximately thirty-five candidate items distributed across four sub-clusters surfaced when the cycle’s re-application of the refined method [4] returned elevated embedding-similarity

scores against multiple existing ControlObjectives without converging on a single primary anchor. The batch was registered as a single ACR candidate to evaluate whether the cycle’s similarity-stage signal had detected genuinely new ontological substance or had surfaced items already covered at appropriate adjacency by existing entities.

Per-condition evidence. - *Condition 1.* Source coverage across the cycle’s substrate met the multi-source threshold at the batch level; however, the batch’s items as a coherent cluster did not converge on a single substantive concept at the candidate level — items distributed across multiple existing ControlObjectives without a unifying concept that the protocol could promote. - *Condition 2.* Multi-stage method signals were *partially* aligned: the embedding-similarity stage flagged elevated scores; the structural-hierarchy stage did not return a coherent slice signal that would have anchored the batch to a single substantive promotion. The override-mechanism record carries the author’s evaluation that the batch’s items, on closer review, were either already covered by existing entities at appropriate adjacency or were items whose substantive content fell outside the bounded ontology’s scope and into the practitioner Manual’s prose layer. - *Condition 3.* Where the batch’s items had practitioner-Manual content backing, the backing was already covered by existing ontology entities at appropriate adjacency, with no residual content motivating new substance. - *Condition 4.* No coherent slice fit emerged — the batch’s items, as a candidate cluster, did not motivate either an existing-slice fit or a defensible slice-contract amendment.

Disposition. Not admitted. The protocol’s threshold is not met: items are either already covered at appropriate adjacency by existing entities, or are in scope for the practitioner Manual rather than for the bounded ontology. The non-admission is reported as an active result of the protocol — the protocol’s symmetric application produces non-promotion under the same conditions as it produces promotion, and the cycle’s record of the non-admission is itself evidence that the threshold is testable rather than retrofitted.

The ACR-003 identifier is consumed: future cycles will not re-use the identifier for a different candidate, even if substance later resurfaces under different framing.

9.2 ACR-004 — Output Rendering Safety / Context-Aware Encoding (promoted)

Substance. Output rendering safety and context-aware encoding for content destined for downstream interpretation (HTML rendering, SQL execution, command-interpreter execution, log-injection contexts, and analogous surfaces). Distinct from input validation alone — the input may be valid in form yet still produce unsafe output if context-appropriate encoding is omitted at the rendering boundary. The substance addresses the output side of the input-and-output-validation slice’s substrate.

v0-era status. As with ACR-002 (§8.2), output rendering safety / context-aware encoding is so foundational to AppSec that its absence as first-class typed substance in v0 deserves explanation. The substance was present in the practitioner Manual’s validation chapter at v0 publication, and v0’s input-validation slice carried adjacent ControlObjectives covering input boundary discipline and controlled failure under hostile input — but the *output side* of the validation slice (context-aware encoding at the rendering boundary) was not first-class typed substance under explicit governance. ACR-004 is therefore a *first-class promotion of pre-existing Manual substance* under the ACR protocol’s discipline, with an additive scope amendment expanding the slice’s contract from “input validation and controlled failure” to “input/output validation and controlled failure”; it is not a discovery of new substance unaccounted for in the v0-era programme.

Per-condition evidence. - *Condition 1.* At promotion time the substance converges across 5 independent sources — OWASP ASVS V5 [10], MITRE CWE-79 [14], MITRE CAPEC-63/242 [13], OWASP MCP

guidance, and OWASP DSOMM [17] — meeting the protocol’s ≥ 3 -source bar for a new-Core-entity promotion. The promotion introduces new Core entities and an additive contract amendment but does not open a new slice, so the ≥ 3 -source burden applies. The per-source claim distribution is preserved in the substrate’s per-pilot configuration files at the cycle-close release tag (see [4 §15.2]). - *Condition 2*. Multi-stage method convergence: the structural-hierarchy stage places the candidate within the IVF slice with the slice-contract amendment proposed; the embedding-similarity stage returns coherent top-anchor cluster across the convergent sources; the override-mechanism record concurs with the method signal. - *Condition 3*. Substance present in the practitioner Manual under the validation chapter of the Manual’s prose layer. - *Condition 4*. The candidate motivates an additive amendment to the IVF slice’s per-slice contract scope, expanding from “input validation and controlled failure” to “input/output validation and controlled failure”. The amendment is additive: prior scope (input validation, controlled failure under hostile or malformed input) is preserved; the candidate adds the output-side substance.

Disposition. Promoted. The substance is admitted to the ontology as one new ControlObjective + one new Practice + one new Mechanism in the (now-expanded) IVF slice, together with the additive amendment to the slice’s per-slice contract scope.

9.3 The four-decision symmetric application

The four-decision span — ACR-001 promoted (§8.1), ACR-002 promoted (§8.2), ACR-003 not admitted (§9.1), ACR-004 promoted (§9.2) — is the evidence supporting Contribution 3. Three promotions and one non-admission decided under the same four-condition threshold provide initial evidence that the protocol produces falsifiable governance decisions: the same conditions that produced promotion under ACR-001, ACR-002, and ACR-004 produced non-admission under ACR-003. The protocol’s threshold is therefore testable rather than retrofitted; the symmetric application is the record against which future ACR decisions can be audited for consistency with the published threshold, and §10.6 registers the empirically grounded near-miss non-admission case as future work to close the discriminative-power evidence against single-condition marginal failures.

10. Limitations

10.1 Independent-research scope

This artefact and its governance protocol are published as independent research [4 §8.3]. The following limits are declared honestly and registered as future work:

- *No subject-matter-expert panel review of individual mappings*. Per-instance mapping decisions and the per-condition evidence for each of the four ACR worked decisions are author-curated under the cycle’s documented discipline [4]. Subject-matter-expert review of individual mappings is not part of the artefact’s scope; future iterations of the cycle may incorporate such review and revise the artefact accordingly.
- *No inter-rater reliability measurement*. The per-instance population of v1, the within-slice refinement attributions of §6.3, and the four ACR worked decisions are single-evaluator categorisations. Inter-rater reliability validation of these categorisations is not part of this paper’s scope.
- *No peer-validated submission of individual mappings*. The cycle’s per-source-into-Reference mapping skin (the OLIR-compatible publication form reported in [4]) is form-level only; peer submission to a programme such as NIST OLIR or to a related cross-mapping community is not part of this paper’s scope and is registered as future work.

10.2 Schema-preservation claim is bounded to the cycle-close corpus

Contribution 2's schema-preservation claim is bounded to the thirty-one-source corpus at the cycle's close. Future source expansion may surface ACR pressure that the protocol's current threshold does not anticipate, or may surface substance that the schema cannot absorb additively under existing slice contracts. The cycle's reusable workflow [4] is the response: re-invocation of the cycle's method on a new source is the discipline by which future expansion is integrated, with the protocol of §7 governing whether new ontology change is warranted. The artefact published here does not claim future-source bounding.

10.3 The bidirectional Manual-validation cycle is downstream

The bidirectional validation cycle that exercises v1 against the practitioner Manual's prose layer — the Manual covering v1's instance population in prose, the Manual's coverage of substance routed outside v1's bounds, the edit-versus-traceback decisioning for each of the four ACR worked decisions — is reported in [5]. Misfit surfaced by [5] may iterate the artefact reported here. The artefact's release at the cycle-close release tag is the artefact's state at the design science cycle's close; revision under [5]'s validation cycle would produce a new release tag.

10.4 EvidencePattern population is deferred

EvidencePattern is preserved as a declarative class in v1 (§2.2) without additional populated instances at this release. The v0 supporting index of evidence patterns [1, §6] is preserved; population of the EvidencePattern class as first-class typed instances is registered as future programme work. Downstream consumers requiring deterministic evidence-pattern verification can use the v0 supporting index against the v1 populated graph at the cycle-close release tag; first-class typed-instance population would extend the artefact's machine-verifiability surface.

10.5 Versioning policy

The artefact follows a semantic-versioning policy aligned with the ACR protocol's governance scope:

- A major version increment (v1 → v2) reflects a partition-level change: addition or removal of a slice, addition of a populated entity type, modification of the cross-slice relation type set, or restructuring of an existing slice's contract beyond additive amendment. Such changes are out of scope of the ACR protocol (§7.2 Condition 4) and require a separate partition-level governance exercise.
- A minor version increment (v1 → v1.1, v1.2) reflects ACR-promoted additive change under the protocol of §7: a promoted ControlObjective / Practice / Mechanism / Artifact set, an additive amendment to an existing slice's contract scope, or a new ACR non-admission decision recorded against the protocol's threshold.
- A patch version increment (v1.0 → v1.0.1, v1.0.2) reflects within-slice refinements admitted under the lighter governance discipline of §6.3 (per-identifier audit + slice-contract consistency + cumulative-volume threshold), or non-substantive editorial revisions (metadata, formatting, controlled-vocabulary fixes) that do not alter populated content or relation declarations.

v1 as published here is the artefact's first OWL/SHACL formal release; v0 [1] was YAML-only. The v0 → v1 transition is therefore an artefact-formalization step that decomposes into the additive instance population documented in §6 (minor-version equivalent: ACR-promoted additions) plus the within-slice 4-Mechanism refinement (patch-version equivalent). Future evolution under the policy above is the artefact's evolution path; partition-level revisions remain subject to separate programme-lead governance outside the ACR protocol's scope.

10.6 Discriminative power against near-miss non-admission is not empirically demonstrated

The protocol’s discriminative power under the four-condition threshold is established by the protocol’s specification (§7.2), and the symmetric application across the four worked decisions of §§8–9 (three promotions, one non-admission) is reported as evidence that the protocol is testable rather than retrofitted. The four worked decisions do not include a *near-miss non-admission* — a candidate that failed a single condition by a marginal amount (for example, a candidate meeting Conditions 2, 3, and 4 but missing Condition 1 by carrying convergent substance across only two sources, one below the three-source bar, or meeting Conditions 1, 2, and 3 but failing Condition 4 by motivating a slice-contract amendment that could not be defended without sibling-slice restructuring). ACR-003’s non-admission was a *framing-level failure* (batch of approximately thirty-five candidate items without a unifying substantive concept across the batch, §9.1), categorically different from a single-condition marginal failure. The three promotions clear the three-source bar (ACR-001 at four sources, ACR-002 and ACR-004 at five, §§8–9), so the cycle exercised admission above the bar but never rejection just below it. The protocol’s discriminative power against single-condition marginal failures is therefore established by the threshold’s specification, not by empirical worked decisions; an empirically grounded near-miss non-admission case is registered as future programme work under the cycle’s reusable workflow [4], to surface in subsequent cycle iterations when a candidate of that profile arises.

Conservative-bias-by-design. The four-condition threshold is calibrated as conservative-by-design: it favours a low false-positive admission rate (promoting candidates that should not be admitted) at the operational cost of a higher false-negative non-admission rate (declining candidates that could perhaps be admitted but fail one condition by a marginal amount). This bias is the defensible governance posture for a bounded ontology where every admission carries downstream cost (apparatus regeneration; MCP grounding refresh; Manual cross-reference update; cross-paper bilateral consistency maintenance across the programme’s papers); under-admission can be reversed by re-promotion under a future cycle, whereas over-admission contaminates the ontology’s bounded surface and the downstream tooling that consumes it. The asymmetric discriminative-power profile of §10.6 is therefore not an empirical gap but a design feature of the protocol’s calibration.

10.7 External-tool validation (OOPS! and FOOPS!)

AppSec Core v1 passes the Ontology Pitfall Scanner OOPS! [35] with zero Critical and zero Important pitfalls. Two Minor pitfalls remain as documented design choices. The first is OOPS! P04 (unconnected ontology element), which affects the abstract grouping superclass `ac:ControlledVocabularyValue`: by design this class carries no direct instance binding (its six controlled-vocabulary subclasses are the instance-bearing classes — the taxonomy-root pattern), and it is connected at class-level via `rdfs:subClassOf` from those six subclasses plus the `owl:disjointWith ac:AppSecCoreEntity` axiom declared in §3.1. The second is OOPS! P22 (different naming conventions), which reflects a deliberate convention split: `snake_case` for schema-derived properties (mirroring canonical YAML keys to keep the OWL surface trivially round-trippable to/from the YAML schema source) and `camelCase` for OWL-generated derived properties (following the established RDF `hasX / isXOf` idiom). Unifying the conventions would require either breaking YAML key fidelity or breaking RDF community readability; both alternatives carry higher cost than the documented split.

AppSec Core v1 scores 13/15 on the FAIR Ontology Pitfall Scanner FOOPS! [36] binary baseline. The two remaining gaps are gated on external prerequisites: PURL1 (persistent URL) pending registration of a `w3id.org` redirect for the ontology namespace, registered as future programme work in the cycle’s reusable workflow [4]; and the OM3 binary check (detailed reusability metadata) pending DOI

assignment at bundle deposit (five of six OM3 sub-fields are already present at the cycle-close release — publisher, logo, status, source, issued — and the missing sub-field is doi). The FOOPS! score will lift to 14/15 when the deposit DOI is assigned and added to the ontology header, and to 15/15 if the `w3id.org` persistent-IRI redirect is subsequently registered.

11. Reproducibility

11.0 The deposited bundle

This paper’s published artefacts are deposited as a single bundle at a public archive (DOI to be assigned at deposit; interim citation form: *cycle-close bundle, archive DOI pending public release*). Within the bundle, the artefacts referenced throughout this paper are identified by stable artefact identifiers — `ontology-v1-final` (the OWL 2 DL ontology + populated graph; §11.1), `apparatus-shacl-pyshacl-v3` (the SHACL apparatus; §11.2), `appsec-core-embeddings-v1.1` (the cycle’s embedding-similarity tooling, used by the cycle reported in [4]) — together with the audit-trail surface that records per-identifier introduction provenance for the `v0` → `v1` entity-count attributions of §6 (Tables 2–3).

Each artefact within the bundle is SHA-256-pinned at the bundle’s manifest. Where this paper cites “the cycle-close release”, the reference is to the bundle deposit; readers verify the SHA-256 pins reported per artefact below against the manifest’s hashes, without requiring access to the cycle’s working repositories.

11.1 OWL 2 DL ontology

The ontology export `ontology-v1-final` is published in Turtle (`.ttl`) format within the bundle, SHA-256-pinned at the manifest. A reader applying a standard OWL 2 DL reasoner (e.g., HermiT, Pellet, OpenLLET) to the populated graph reproduces the class-hierarchy and property-domain/range checks reported in §3.

11.2 SHACL apparatus

The apparatus `apparatus-shacl-pyshacl-v3` is published as two complementary shape files within the bundle: the schema-derived ontology shapes file (`appsec-core-v0-shapes.ttl`) and the consumer-conformance shapes file (`consumer-conformance-shapes.ttl`), together with the build script (`build_shacl.py`) for the schema-derived shapes and its documented build environment. Re-running the build script under the documented environment produces a deterministic SHACL output reproducible against the released schema-derived shapes file (isomorphic up to blank node labeling under the RDFLib serialisation defaults of the documented environment); SHA-256 pinning at the bundle’s manifest is the canonical reference for all three files.

11.3 Validation pair

The validation pair is `pyshacl` version 0.31.0 with `rdfLib` 7.6.0 (the SHACL Core reference implementation used here as canonical validation oracle, community-maintained against the W3C SHACL Recommendation rather than W3C-endorsed) plus the in-house bounded-subset validator at parity for the apparatus’s six ontology shapes. A reader applying `pyshacl` 0.31.0 + `rdfLib` 7.6.0 to the populated graph + composed shapes graph reproduces the validation outcome: `sh:conforms = true`; zero `sh:Violation`. The bounded-subset validator’s parity for the six ontology shapes is verified at the bundle’s release time and preserved at the bundle’s manifest.

11.4 Bundle artefact identifiers

The bundle’s coordinated release uses the following stable artefact identifiers, each SHA-256-pinned at the manifest:

- `ontology-v1-final` — the OWL 2 DL ontology export with populated graph (§11.1);
- `apparatus-shacl-pyshacl-v3` — the SHACL apparatus shape files + build script (§11.2);
- `appsec-core-embeddings-v1.1` — the cycle’s embedding-similarity tooling, used by the cycle reported in [4] and reproducible from the published embedding model and augmentation rule v1.0. The deposited embedding tensor is a 384-dimensional **float32** L2-normalized NPZ tensor over a 212-entity augmented text corpus (the four populated entity types’ identifier-keyed text representations) under the encoder `sentence-transformers/all-MiniLM-L6-v2` [34] (corpus SHA-256 5951fd82e4b7...; embedding tensor SHA-256 17f6aac496b9...; full 64-char SHA-256 digests recorded in the bundle manifest at the cycle-close release tag).

The bundle’s audit-trail surface records the per-identifier introduction provenance for each of the 25 net additions reported in §6 (Tables 2–3), with one verification entry per identifier and one cross-reference per relation declaration.

11.5 Per-instance attribution and release notes

The bundle’s release notes document per-instance attribution for each of the four ACR worked decisions (ACR-001, ACR-002, ACR-004 promotions and ACR-003 non-admission) and for the within-slice refinement additions (§6.3). A reader auditing the v0→v1 transition against [1]’s published surface can reconstruct the +25 net delta (§6.2) from the per-decision evidence in §§8 and 9 plus the per-identifier introduction record in the audit-trail surface (§11.4).

AI Use Statement

Generative-language-model tools were used by the author during the preparation of this paper for drafting and editing of prose, under the author’s review at every step. All scientific claims, methodological decisions, evidence interpretation, and the per-identifier attribution of the v0→v1 schema-preservation evidence (§6) are the author’s responsibility. The artefact this paper publishes — AppSec Core v1, its OWL 2 DL ontology export (§3), and the SHACL apparatus (§4) — was produced through the design science cycle reported in [4], whose substrate-generation pipeline invokes only a deterministic encoder language model (Sentence-BERT-style sentence embeddings under `sentence-transformers/all-MiniLM-L6-v2` [34]) at runtime; no generative language model is invoked in the cycle’s pipeline execution. SHACL validation under `pyshacl 0.31.0 + rdflib 7.6.0` (§4.2) is deterministic.

12. References

References [4] and [5] are forthcoming programme papers; at the time of this paper’s deposit they are canonical via the construction tags noted in the entry. OSF DOIs are assigned at submission and supersede the construction-tag form.

[1] P. Farinha. *AppSec Core: A Normalized Ontology for Security Requirements Across Heterogeneous Frameworks*. Independent research preprint, 2026. DOI: 10.17605/OSF.IO/WG8PV.

[2] P. Farinha. *Coverage-Preserving Compilation of Normative and Empirical Security Knowledge*. Independent research preprint, 2026. DOI: 10.17605/OSF.IO/A6ZFJ.

- [3] P. Farinha. *Research Programme Prospectus*. Independent research preprint, 2026. DOI: 10.17605/OSF.IO/7T849.
- [4] P. Farinha. *Pressure-Testing AppSec Core: A Design Science Cycle for Bounded-Ontology Evolution Under Heterogeneous Application-Security Sources*. Independent research preprint (companion to this paper), 2026. DOI: 10.17605/OSF.IO/3E8G5. Canonical via construction tag v2.0.0-construction-p7-final-draft.
- [5] P. Farinha. *Coverage-Preserving Compilation v2: A 31-Source Pipeline with Joint Manual and Knowledge-Graph Production*. Independent research preprint (succeeds [2] at 31-source scale), 2026. DOI: 10.17605/OSF.IO/TXW8P. Canonical via construction tag v2.0.0-construction-p8-bundle-complete.
- [6] P. Farinha. *Ontology-Grounded Retrieval for Auditable LLM-Assisted Secure Code Generation*. Independent research preprint, 2026. DOI: 10.17605/OSF.IO/S3HET.
- [7] P. Farinha. *Empirical Evaluation: Deterministic Grounding versus Retrieval-Augmented Generation versus Ungrounded Compilation*. Pre-registered design, 2026 [empirical execution in preparation]. DOI: 10.17605/OSF.IO/H5AJE.
- [8] P. Farinha. *MCP Dual-Mode Instrument Specification*. Independent research preprint, 2026. DOI: 10.17605/OSF.IO/KH8Y7.
- [9] National Institute of Standards and Technology. *Secure Software Development Framework (SSDF) Version 1.1*. NIST Special Publication 800-218, 2022.
- [10] OWASP Foundation. *OWASP Application Security Verification Standard (ASVS) Version 5.0*. 2025. URL: <https://owasp.org/www-project-application-security-verification-standard/>.
- [11] Open Source Security Foundation. *Supply-chain Levels for Software Artifacts (SLSA) Specification v1.0*. 2024. URL: <https://slsa.dev/spec/v1.0/>.
- [12] Center for Internet Security. *CIS Critical Security Controls Version 8.1.2*. 2024. URL: <https://www.cisecurity.org/controls/v8-1>.
- [13] MITRE Corporation. *Common Attack Pattern Enumeration and Classification (CAPEC), version 3.9*. URL: <https://capec.mitre.org/>, accessed 2026.
- [14] MITRE Corporation. *Common Weakness Enumeration (CWE): Software Development View, version 4.19.1*. URL: <https://cwe.mitre.org/>, accessed 2026.
- [15] OWASP Foundation. *OWASP Guidance on Model Context Protocol Security*. 2025 (referenced edition).
- [16] OWASP Foundation. *OWASP Software Assurance Maturity Model (SAMM) Version 2.1*. 2023. URL: <https://owaspsamm.org/>.
- [17] OWASP Foundation. *OWASP DevSecOps Maturity Model (DSOMM)*. 2024. URL: <https://dsomm.owasp.org/>.
- [18] World Wide Web Consortium. *Shapes Constraint Language (SHACL)*. W3C Recommendation, 2017. URL: <https://www.w3.org/TR/shacl/>.
- [19] World Wide Web Consortium. *OWL 2 Web Ontology Language Document Overview (Second Edition)*. W3C Recommendation, 2012. URL: <https://www.w3.org/TR/owl2-overview/>.
- [20] A. R. Hevner, S. T. March, J. Park, and S. Ram. “Design Science in Information Systems Research”. *MIS Quarterly*, 28(1):75–105, 2004.

- [21] R. J. Wieringa. *Design Science Methodology for Information Systems and Software Engineering*. Springer, 2014.
- [22] P. Ralph et al. *ACM SIGSOFT Empirical Standards*. arXiv:2010.03525, 2021.
- [23] N. Guarino. “Formal Ontology and Information Systems”. In *Proceedings of FOIS 1998 — 1st International Conference on Formal Ontologies in Information Systems*, pp. 3–15. IOS Press, 1998.
- [24] Z. Syed, A. Padia, T. Finin, L. Mathews, and A. Joshi. “UCO: A Unified Cybersecurity Ontology”. In *AAAI Workshop on Artificial Intelligence for Cyber Security*, 2016.
- [25] M. D. Iannacone, S. Bohn, G. Nakamura, J. Gerth, K. Huffer, R. A. Bridges, E. M. Ferragut, and J. R. Goodall. “Developing an Ontology for Cyber Security Knowledge Graphs”. In *Proceedings of the 10th Annual Cyber and Information Security Research Conference (CISRC)*, 2015.
- [26] C. Boyer, M. Dogdu, A. Choupani, R. Watson, A. Sanchez, and B. Ametu. “Toward a Unified Cybersecurity Knowledge Graph: Leveraging Ontologies and Open Data Sources”. In *IEEE International Conference on Smart Data Services (SDSC)*, 2024.
- [27] M. C. Suárez-Figueroa, A. Gómez-Pérez, and M. Fernández-López. “The NeOn Methodology framework: A scenario-based methodology for ontology development”. *Applied Ontology*, 10(2):107–145, 2015.
- [28] M. Poveda-Villalón, A. Fernández-Izquierdo, M. Fernández-López, and R. García-Castro. “LOT: An industrial oriented ontology engineering framework”. *Engineering Applications of Artificial Intelligence*, 111:104755, 2022.
- [29] K. Peffers, T. Tuunanen, M. A. Rothenberger, and S. Chatterjee. “A Design Science Research Methodology for Information Systems Research”. *Journal of Management Information Systems*, 24(3):45–77, 2007.
- [30] B. Cuenca Grau, I. Horrocks, Y. Kazakov, and U. Sattler. “Modular Reuse of Ontologies: Theory and Practice”. *Journal of Artificial Intelligence Research*, 31:273–318, 2008.
- [31] B. Konev, C. Lutz, D. Walther, and F. Wolter. “Formal Properties of Modularisation”. In *Modular Ontologies* (Springer LNCS 5445), pp. 25–66, 2009.
- [32] N. Keller, P. Quinn, and M. Smith. *Mapping Relationships Between Documentary Standards, Regulations, Frameworks, and Reference Materials*. NIST Internal Report 8477, 2024.
- [33] N. Keller, J. Barrett, P. Quinn, and M. Smith. *National Online Informative References (OLIR) Program: Submission Guidance for OLIR Developers*. NIST Internal Report 8278A Revision 1, 2021.
- [34] N. Reimers and I. Gurevych. “Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks”. In *Proceedings of EMNLP-IJCNLP 2019*, pp. 3982–3992. arXiv:1908.10084.
- [35] M. Poveda-Villalón, A. Gómez-Pérez, and M. C. Suárez-Figueroa. “OOPS! (Ontology Pitfall Scanner!): An On-line Tool for Ontology Evaluation”. *International Journal on Semantic Web and Information Systems*, 10(2):7–34, 2014. DOI: 10.4018/ijswis.2014040102. URL: <https://oops.linkeddata.es/>.
- [36] D. Garijo, O. Corcho, and M. Poveda-Villalón. “FOOPS!: An ontology pitfall scanner for the FAIR principles”. In *Proceedings of the ISWC 2021 Posters, Demos and Industry Tracks*, CEUR Workshop Proceedings Vol. 2980, 2021. URL: <https://foops.linkeddata.es/>.
- [37] National Institute of Standards and Technology. *Security and Privacy Controls for Information Systems and Organizations*. NIST Special Publication 800-53 Revision 5, 2020. DOI: 10.6028/NIST.SP.800-53r5.

- [38] PCI Security Standards Council. *Payment Card Industry Data Security Standard (PCI DSS) Version 4.0.1*, June 2024. URL: https://www.pcisecuritystandards.org/document_library/.
- [39] PCI Security Standards Council. *Payment Card Industry Software Security Standard — Secure Software Lifecycle (PCI SSLC) Requirements Version 1.1*, 2022. URL: https://www.pcisecuritystandards.org/document_library/.
- [40] European Parliament and Council. *Regulation (EU) 2024/2847 on horizontal cybersecurity requirements for products with digital elements (Cyber Resilience Act, CRA)*, October 2024. URL: <https://eur-lex.europa.eu/eli/reg/2024/2847>.
- [41] European Parliament and Council. *Directive (EU) 2022/2555 on measures for a high common level of cybersecurity across the Union (NIS2 Directive)*, December 2022. URL: <https://eur-lex.europa.eu/eli/dir/2022/2555>.
- [42] European Parliament and Council. *Regulation (EU) 2016/679 (General Data Protection Regulation, GDPR)*, April 2016. URL: <https://eur-lex.europa.eu/eli/reg/2016/679>.
- [43] European Parliament and Council. *Regulation (EU) 2022/2554 on digital operational resilience for the financial sector (DORA)*, December 2022. URL: <https://eur-lex.europa.eu/eli/reg/2022/2554>.
- [44] U.S. Department of Health and Human Services. *Health Insurance Portability and Accountability Act (HIPAA) Security Rule*, 45 CFR Parts 160 and 164, Subparts A and C. URL: <https://www.hhs.gov/hipaa/for-professionals/security/>.
- [45] National Institute of Standards and Technology. *Adversarial Machine Learning: A Taxonomy and Terminology of Attacks and Mitigations*. NIST AI 100-2 E2025, 2025. DOI: 10.6028/NIST.AI.100-2e2025.
- [46] National Institute of Standards and Technology. *Artificial Intelligence Risk Management Framework (AI RMF 1.0)*. NIST AI 100-1, January 2023. DOI: 10.6028/NIST.AI.100-1.
- [47] OWASP Foundation. *OWASP Proactive Controls*, 2018 edition. URL: <https://owasp.org/www-project-proactive-controls/>.
- [48] OWASP Foundation. *OWASP Top 10 for Large Language Model Applications (LLM01–LLM10)*, 2025. URL: <https://genai.owasp.org/llm-top-10/>.
- [49] MITRE Corporation. *Adversarial Threat Landscape for Artificial-Intelligence Systems (ATLAS) v5.6.0*. URL: <https://atlas.mitre.org/>.
- [50] SAFECODE. *Practices for Secure Development of Cloud Applications; Fundamental Practices for Secure Software Development; Software Integrity Controls* (SAFECODE Agile 2012, FPSSD 2018, SIC 2010). URL: <https://safecode.org/publications/>.